

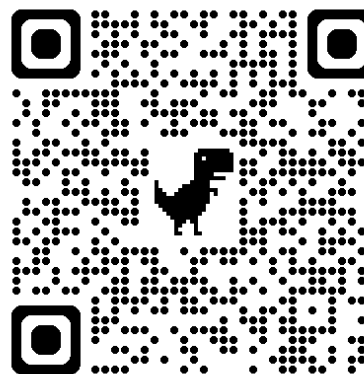


# 自然語言處理與應用

# Natural Language Processing and Applications

**Hugging Face Tutorial**

Instructor: 林英嘉 (Ying-Jia Lin)  
2026/04/27



GitHub



Slido  
# NLP0427  
([Link](#))

# What you will learn in this tutorial

---

- Training and evaluating GPT-2 / T5 on abstractive summarization
  - Dataset: Chinese abstractive summarization
  - Objective: Cross-entropy
  - Main packages: PyTorch, Hugging Face, ROUGE

# Why do we need to learn GPT-2?

---

- GPT-2 is the **last** open-source model in the GPT series from OpenAI.
- Language generation is hard, and GPT-2 is a good start point.
- GPT-2 is not big. It is feasible for low-budget machines.
  - GPT-2 (12-layer; 124M), GPT-2-medium (24-layer; 345M), GPT-2-large (36-layer; 762M), GPT-2-xl (48-layer; 1.5B)

# Introduction to Text Summarization

---

- Extractive summarization
  - Generate a short text summary for a document by **selecting salient sentences** in the documents.
- Abstractive summarization
  - Generate **novel words** and phrases not featured in the source text.



## Example:

The Queen's Birthday holiday road toll stands at zero for the first time since records began.

**Ext summary:** The Queen's holiday road toll stands at zero.

**Abs summary:** The Queen's holiday slashes road toll.

# Task: Chinese Abstractive Summarization

---

- Dataset: LCSTS (A Large-Scale Chinese Short Text Summarization Dataset)<sup>[1]</sup>

**Short Text:** 水利部水资源司司长陈明忠今日在新闻发布会上透露，根据刚刚完成的水资源管理制度的考核，有部分省接近了红线的指标，有部分省超过红线的指标。在一些超过红线的地方，将对一些取用水项目进行区域的限批，严格地进行水资源论证和取水许可的批准。

**Summarization:** 部分省超过年度用水红线指标 取水项目将被限批

[1] Hu, Baotian, Qingcai Chen, and Fangze Zhu. "LCSTS: A Large Scale Chinese Short Text Summarization Dataset." Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. 2015.

# 今日著重在 Hugging Face Transformers

## Deployment & Inference

<https://huggingface.co/docs>

### • Inference Providers

Call 200k+ models hosted by our 10+ Inference partners

### • Inference Endpoints (dedicated)

Deploy models on dedicated & fully managed infrastructure on HF

### • Deploying on AWS

Train/deploy models from Hugging Face to AWS with DLCs

### • Text Generation Inference

Serve language models with TGI optimized toolkit

### • Text Embeddings Inference

Serve embeddings models with TEI optimized toolkit

### • Microsoft Azure

Deploy Hugging Face models on Microsoft Azure

### • Google Cloud

Train and Deploy Hugging Face models on Google Cloud

## Core ML Libraries

### • Transformers

State-of-the-art AI models for PyTorch

### • Diffusers

State-of-the-art Diffusion models in PyTorch

### • Datasets

Access & share datasets for any ML tasks

### • Transformers.js

State-of-the-art ML running directly in your browser

### • Tokenizers

Fast tokenizers optimized for research & production

### • Evaluate

Evaluate and compare models performance

### • timm

State-of-the-art vision models: layers, optimizers, and utilities

### • Sentence Transformers

Embeddings, Retrieval, and Reranking

### • Kernels

Load and run compute kernels from the Hugging Face Hub

## Training & Optimization

### • PEFT

Parameter-efficient finetuning for large language models

### • Accelerate

Train PyTorch models with multi-GPU, TPU, mixed precision

### • Optimum

Optimize HF Transformers for faster training/inference

### • AWS Trainium & Inferentia

Train/deploy Transformers/Diffusers on AWS

### • Google TPUs

Train and Deploy models on Google TPUs via Optimum.

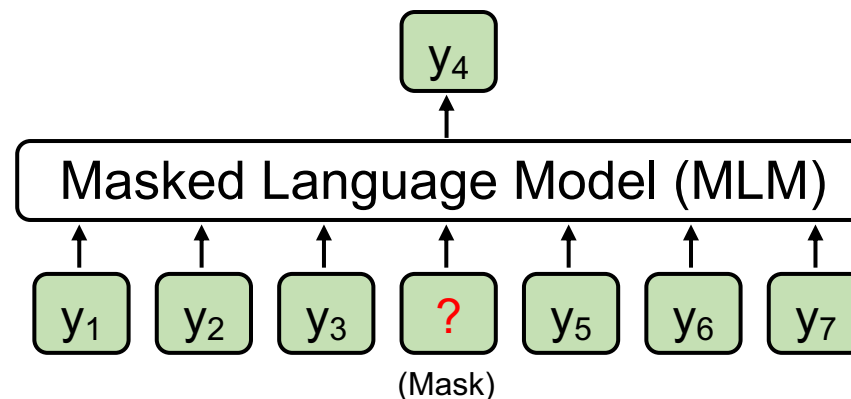
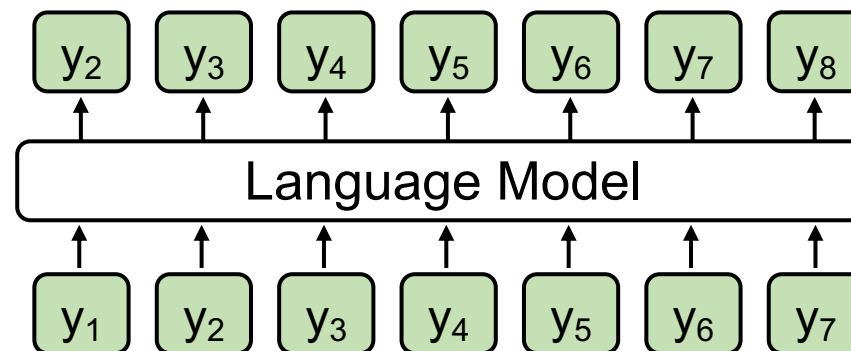
### • TRL

Train transformers LMs with reinforcement learning

# Language Model vs. Masked Language Model

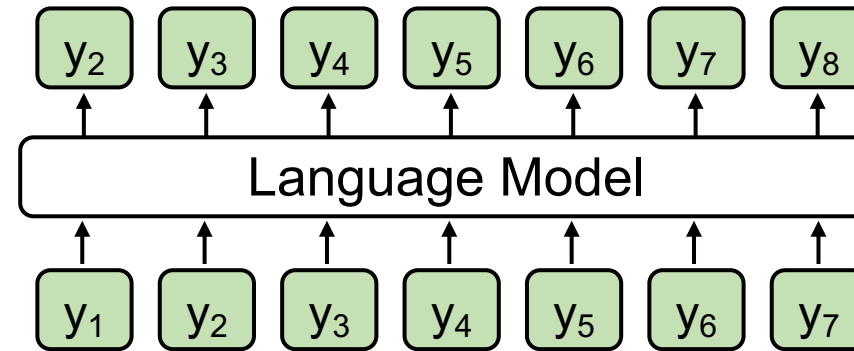
Casual Language Model  
(E.g., GPT)

Masked Language Model  
(E.g., BERT)

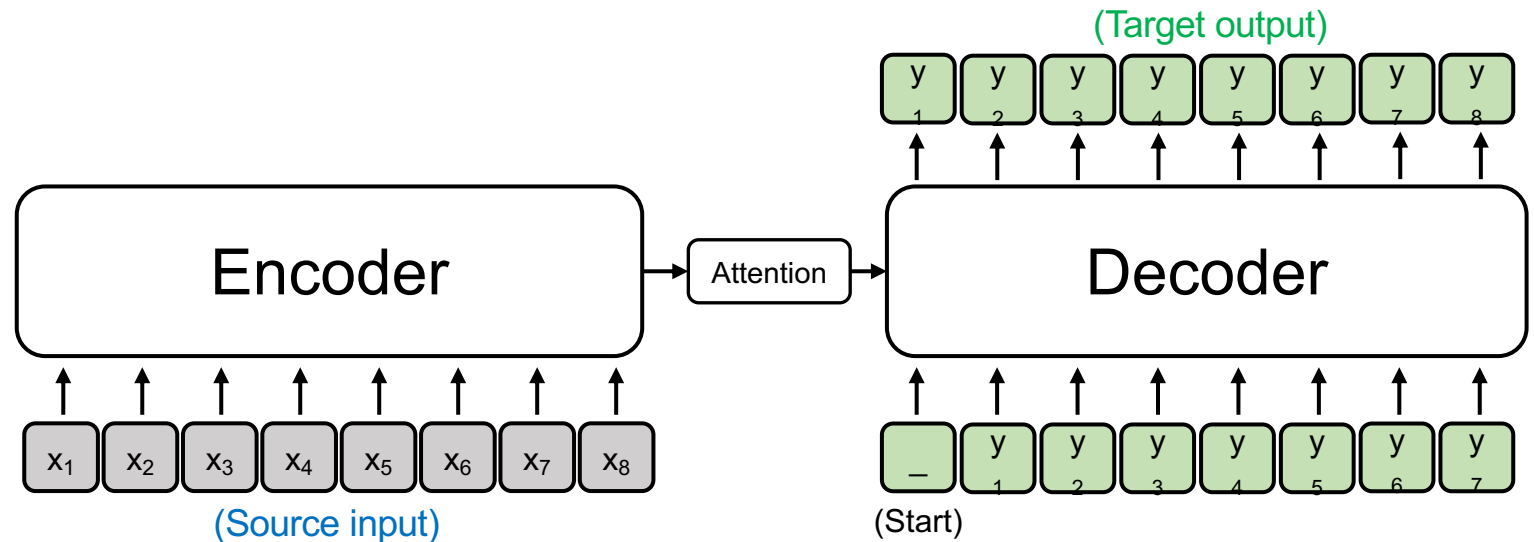


# Language Model vs. Seq2seq Model

Casual Language Model  
(E.g., GPT)



Sequence to sequence Model  
(E.g., T5; Vanilla Transformers)





# Load LCSTS via Hugging Face Dataset

---

- [Hugging Face dataset link](#)

```
raw_train = load_dataset(  
    "hugcyp/LCSTS", split="train", cache_dir="./cache/"  
)  
.to_list()  
raw_val = load_dataset(  
    "hugcyp/LCSTS", split="validation", cache_dir="./cache/"  
)  
.to_list()
```

Sometimes checking the Hugging Face dataset is slow, it will be faster if we transform the dataset object into a list using `.to_list()`.

# Let's try it

---

1. 用 `load_dataset()` 把 train 和 validation 分開讀進來
2. 把 dataset 轉成 pandas dataframe，觀察 1~2 筆 samples

# Auto Classes (models, tokenizers)

---

- In many cases, the architecture you want to use **can be guessed** from the name or the path of the pretrained model you are supplying to the `from_pretrained` method.
- AutoClasses are here to do this job for you so that you automatically retrieve the relevant model given the name/path to the pretrained weights/config/vocabulary:
  - Ex: `model = AutoModel.from_pretrained('bert-base-cased')` will create an instance of the BERT base model (cased).

# Generative Models in Auto Classes

---

Casual Language Model  
(E.g., GPT, Llama)

AutoModelForCausalLM ([link](#))  
GPT2LMHeadModel ([link](#))  
LlamaForCausalLM ([link](#))

Masked Language Model  
(E.g., BERT)

AutoModelForMaskedLM ([link](#))  
BertForMaskedLM ([link](#))

Sequence to sequence Model  
(E.g., T5; Vanilla  
Transformers)

AutoModelForSeq2SeqLM ([link](#))  
T5ForConditionalGeneration ([link](#))

# Outline

---

- GPT-2 (native PyTorch)
- ~~T5 (Hugging Face Dataset and Trainer)~~

# Load the Tokenizer and the Model

---

- [Hugging Face model link](#)

```
model_name = "uer/gpt2-chinese-cluecorpussmall"  
tokenizer = AutoTokenizer.from_pretrained(  
    model_name,  
    padding_side="left",  
)  
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})  
model.resize_token_embeddings(len(tokenizer))
```

Optional

# Left padding

In text generation, sometimes we set `padding_side='left'`, when loading tokenizer.

If padding is at the end of the prompt, new tokens may be generated after the padding, which is illogical.



|     |        |     |       |       |           |       |       |       |  |
|-----|--------|-----|-------|-------|-----------|-------|-------|-------|--|
| <s> | Recite | the | first | law   | <s>       | <pad> | <pad> | <pad> |  |
| <s> | How    | are | you   | <s>   | <pad>     | <pad> | <pad> | <pad> |  |
| <s> | Who    | is  | the   | first | president | of    | U.S.  | <s>   |  |



Left-padding

Aligning the new tokens.



|       |       |       |       |        |           |       |      |     |  |
|-------|-------|-------|-------|--------|-----------|-------|------|-----|--|
| <pad> | <pad> | <pad> | <s>   | Recite | the       | first | law  | <s> |  |
| <pad> | <pad> | <pad> | <pad> | <s>    | How       | are   | you  | <s> |  |
| <s>   | Who   | is    | the   | first  | president | of    | U.S. | <s> |  |

# About Padding

---

|     | Fine-tuning                                                | Test time (inference)        |
|-----|------------------------------------------------------------|------------------------------|
| 策略一 | Left padding + set [PAD] as -100 in labels (this tutorial) | Left padding (this tutorial) |
| 策略二 | Right padding + set [PAD] as -100 in labels                | Use test_batch_size as 1     |

- Fine-tuning 的策略一可搭配 Test time 的任一策略
- Fine-tuning 的策略二只建議搭配 Test time 的策略二



# Set [PAD] as -100 in labels

---

[https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling\\_gpt2.py#L1264-L1267](https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py#L1264-L1267)

```
"""
labels (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*):
    Labels for language modeling. Note that the labels **are shifted** inside the model, i.e. you can set
    `labels = input_ids` Indices are selected in `[-100, 0, ..., config.vocab_size]` All labels set to `-100`
    are ignored (masked), the loss is only computed for labels in `[0, ..., config.vocab_size]`
"""
```

# Load the Tokenizer and the Model

---

- [Hugging Face model link](#)

```
model_name = "uer/gpt2-chinese-cluecorpussmall"  
tokenizer = AutoTokenizer.from_pretrained(  
    model_name,  
    padding_side="left",  
)  
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})  
model.resize_token_embeddings(len(tokenizer))
```

Increasing the size will add newly initialized vectors at the end.  
Reducing the size will remove vectors from the end. ([link](#))

# resize\_token\_embeddings

---

- 新增 embedding 的情況：
  - 新增的 embedding 還沒有被訓練過
  - 透過 torch 的 nn.Embedding 來新增 token 的向量
  - [https://github.com/huggingface/transformers/blob/v4.51.3/src/transformers/modeling\\_utils.py#L2802-L2807](https://github.com/huggingface/transformers/blob/v4.51.3/src/transformers/modeling_utils.py#L2802-L2807)
  - torch 的 nn.Embedding 的初始化範圍是 normal distribution 0到1之間

# Let's try it

---

1. 載入 tokenizer
2. 測試句子：「今天天氣真好」，print 出 tokenized tokens

# Create the PyTorch Dataset

- You can use the Hugging Face dataset without building a PyTorch dataset class.
- However, language generation is complicated. We suggest building your first code with native PyTorch.

```
1 class LCSTSDataset(torch.utils.data.Dataset):
2     def __init__(self, raw_data) -> None:
3         super().__init__()
4         self.data = raw_data
5         self.token_replacement = [
6             [":", ":"],
7             [",", ","],
8             ["'", "'"],
9             ["'", "'"],
10            ["?", "?"],
11            ["...", "..."],
12            ["!", "!"],
13        ]
14
15    def __getitem__(self, index):
16        d = self.data[index]
17        # Substitute some full-width punctuations with half-width ones
18        for k in d:
19            for tok in self.token_replacement:
20                d[k] = d[k].replace(tok[0], tok[1])
21        return d
22
23    def __len__(self):
24        return len(self.data)
```

To prevent out-of-vocabulary tokens from being transformed into [UNK]

Run:

```
train_set = LCSTSDataset(raw_train)
val_set = LCSTSDataset(raw_val)
```

# Create the PyTorch DataLoader for Data Batching

---

1. Input data for training (source + summary)
2. Labels (auto-regressive; thus, basically input\_ids themselves)
3. Input data for predictions (source only)

# Overview

```
1 def collate_fn(batch):
2     complete_text = [
3         f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
4         for example in batch
5     ]
6     complete_text = tokenizer(
7     1     complete_text,
8         padding=True,
9         truncation=True,
10        return_tensors="pt",
11        add_special_tokens=False,
12    )
13    # Set label padding tokens to -100 for loss masking
14    labels = torch.where(
15        condition=complete_text.input_ids != tokenizer.pad_token_id,
16        input=complete_text.input_ids,
17    2        other=-100,
18    )
19    complete_text["labels"] = labels
20    complete_text = {k: complete_text[k].to(device) for k in complete_text}
21
22    infer_text = [example["text"] for example in batch]
23    infer_text = tokenizer(
24        infer_text,
25    3        padding=True,
26        truncation=True,
27        return_tensors="pt",
28    )
29    infer_text = {k: infer_text[k].to(device) for k in infer_text}
30    return complete_text, infer_text
```

# Create the PyTorch DataLoader for Data Batching: 1. Input Data for Training

```
1 def collate_fn(batch):
2     complete_text = [
3         f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
4         for example in batch
5     ]
6     complete_text = tokenizer.batch_encode_plus(
7         complete_text,
8         padding=True,
9         truncation=True,
10        return_tensors="pt",
11        add_special_tokens=False,
12    )
```

You can omit the [CLS] token before fine-tuning!





# uer/gpt2-chinese-cluecorpussmall uses BertTokenizer

<https://huggingface.co/uer/gpt2-chinese-cluecorpussmall/blob/main/config.json#L26>

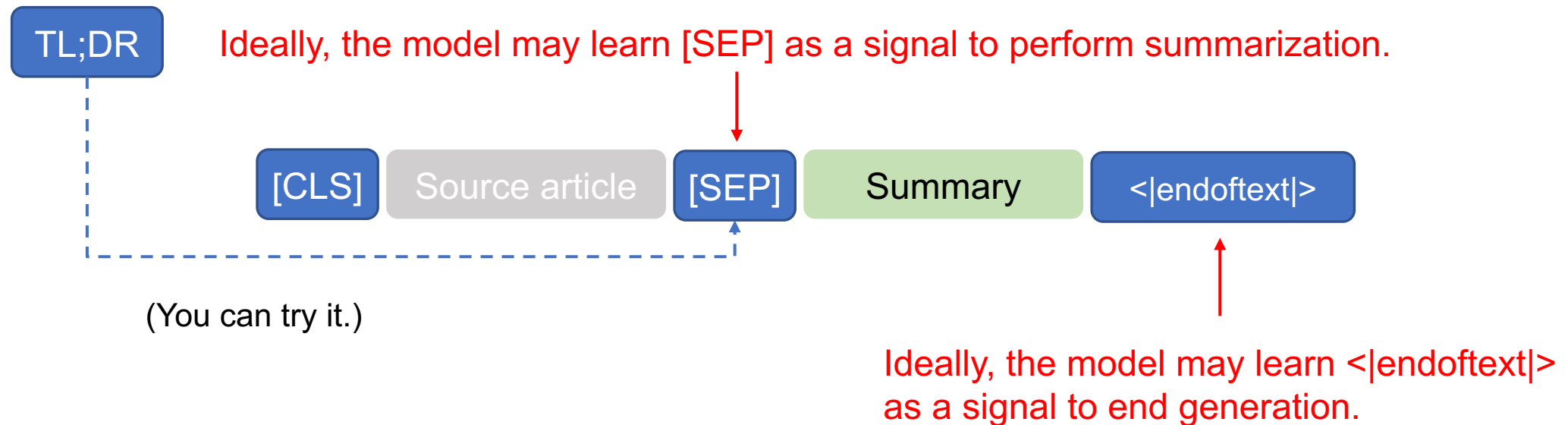
```
20     "task_specific_params": {  
21         "text-generation": {  
22             "do_sample": true,  
23             "max_length": 320  
24         }  
25     },  
26     "tokenizer_class": "BertTokenizer",  
27     "vocab_size": 21128
```

<https://huggingface.co/ckiplab/gpt2-base-chinese/blob/main/config.json#L32>

```
26     "task_specific_params": {  
27         "text-generation": {  
28             "do_sample": true,  
29             "max_length": 50  
30         }  
31     },  
32     "tokenizer_class": "BertTokenizerFast",  
33     "vocab_size": 21128
```

# English GPT-2 for Text Summarization<sup>[2]</sup>

> To induce summarization behavior, we can add the text TL;DR: after the article <sup>[2]</sup>



[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI blog, 1(8), 9.

# Create the PyTorch DataLoader for Data Batching: 2. Labels (auto-regressive)

(We are still at the `collate_fn` function.)

```
13 # Set label padding tokens to -100 for loss masking
14 labels = torch.where(
15     condition=complete_text.input_ids != tokenizer.pad_token_id,
16     input=complete_text.input_ids,
17     other=-100,
18 )
19 complete_text["labels"] = labels
20 complete_text = {k: complete_text[k].to(device) for k in complete_text}
```

this is sentence a

|        |       |       |      |     |          |     |
|--------|-------|-------|------|-----|----------|-----|
|        | [PAD] | [PAD] | this | is  | sentence | a   |
| labels | -100  | -100  | 1212 | 318 | 6827     | 317 |

this is is is sentence b

|        |      |     |     |     |          |     |
|--------|------|-----|-----|-----|----------|-----|
|        | this | is  | is  | is  | sentence | b   |
| labels | 1212 | 318 | 318 | 318 | 6827     | 347 |

# GPT2LMHeadModel shifts logits and labels

[https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling\\_gpt2.py#L1300-L1301](https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py#L1300-L1301)

Shape of lm\_logits: [batch\_size, seq\_length, hidden\_size]

```
1296         if labels is not None:
1297             # move labels to correct device to enable model parallelism
1298             labels = labels.to(lm_logits.device)
1299             # Shift so that tokens < n predict n
1300             shift_logits = lm_logits[..., :-1, :].contiguous()
1301             shift_labels = labels[..., 1:].contiguous()
1302             # Flatten the tokens
1303             loss_fct = CrossEntropyLoss()
1304             loss = loss_fct(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
```

:-1 代表取到倒數第二個  
1: 代表答案後移一格

.contiguous():

[https://youtu.be/FGJSGAt\\_9Ks?si=GMxxS2Oi742e2Bhl&t=1261](https://youtu.be/FGJSGAt_9Ks?si=GMxxS2Oi742e2Bhl&t=1261)

# Create the PyTorch DataLoader for Data Batching: 3. Input Data for Predictions

(We are still at the `collate_fn` function.)

We will use ``infer_text`` during **evaluations**.

```
22     infer_text = [example["text"] for example in batch]
23     infer_text = tokenizer.batch_encode_plus(
24         infer_text,
25         padding=True,
26         truncation=True,
27         return_tensors="pt",
28     )
29     infer_text = {k: infer_text[k].to(device) for k in infer_text}
30     return complete_text, infer_text
```

If you omit the [CLS] token during fine-tuning, then you don't need to add [CLS] before evaluations!



# Create the PyTorch DataLoader for Data Batching: 1. Input Data for Training [Recap]

```
1 def collate_fn(batch):
2     complete_text = [
3         f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
4         for example in batch
5     ]
6     complete_text = tokenizer.batch_encode_plus(
7         complete_text,
8         padding=True,
9         truncation=True,
10        return_tensors="pt",
11        add_special_tokens=False,
12    )
```

You can omit the [CLS] token before fine-tuning!



# Create the PyTorch DataLoader for Data Batching

---

```
1 train_loader = torch.utils.data.DataLoader(  
2     train_set,  
3     batch_size=TRAIN_BATCH_SIZE,  
4     shuffle=True, ← Prevent a model from overfitting on data order  
5     collate_fn=collate_fn,  
6 )  
7 val_loader = torch.utils.data.DataLoader(  
8     val_set,  
9     batch_size=VAL_BATCH_SIZE,  
10    shuffle=False,  
11    collate_fn=collate_fn,  
12 )
```

We don't need to use the test set because there is no public test labels (summaries).

See <https://huggingface.co/datasets/hugcyp/LCSTS/viewer/default/test>

# Set up Optimizer

---

```
optimizer = torch.optim.AdamW(model.parameters(), lr=LR)
```

- **optimizers** (Tuple[torch.optim.Optimizer, torch.optim.lr\_scheduler.LambdaLR], *optional*, defaults to (None, None)) — A tuple containing the optimizer and the scheduler to use. Will default to an instance of **AdamW** on your model and a scheduler given by `get_linear_schedule_with_warmup()` controlled by `args`.

Trainer

**Trainer**

Seq2SeqTrainer

TrainingArguments

Seq2SeqTrainingArguments



# Let's try it

---

1. 開 train 一下模型

# Set up Evaluation Metric

```
rouge_metric = Rouge()
```

The screenshot shows the GitHub repository page for `pltrdy / rouge`. The repository is public and has 8 watches, 100 forks, and 668 stars. The main content area displays a list of files and their commit history:

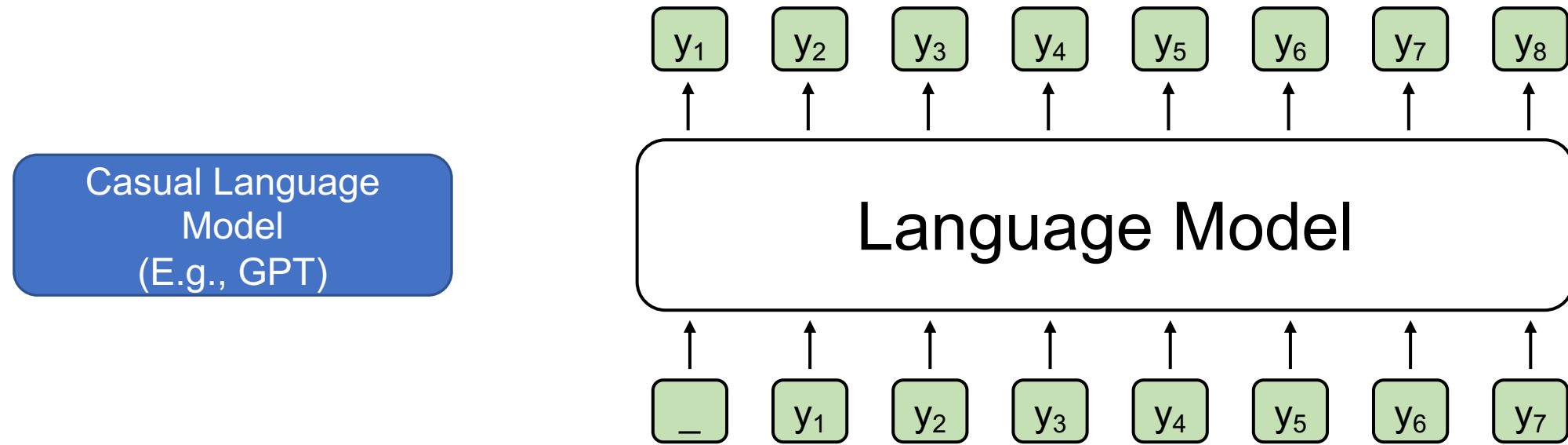
| File        | Commit Message                                 | Time Ago    |
|-------------|------------------------------------------------|-------------|
| bin         | Exclusive option + raw results                 | 5 years ago |
| rouge       | bump version 1.0.1                             | 3 years ago |
| tests       | Merge pull request #35 from pltrdy/update_f... | 5 years ago |
| .gitignore  | Introducing ROUGE: A full Python Implement...  | 7 years ago |
| LICENSE     | Initial commit                                 | 7 years ago |
| MANIFEST.in | 0.2.1: python2 support                         | 7 years ago |
| README.md   | Update README.md                               | 3 years ago |
| setup.py    | minor setup.py fix                             | 4 years ago |

On the right side, the 'About' section describes the repository as 'A full Python Implementation of the ROUGE Metric (not a wrapper)'. It also lists the README, Apache-2.0 license, and activity. The 'Releases' section shows the latest release, `rouge 1.0.1: results on par wit...`, dated Feb 18, 2020, marked as 'Latest'.

<https://github.com/pltrdy/rouge>

# Generating step-by-step

---



- Training time: update the model through hidden states
- Test time: perform **decoding** for each step

# Decoding strategies

---

- Popular decoding strategies:
  - Greedy decoding
  - Beam search
  - Top-k sampling
  - Top-p sampling

# How to let a model generate step-by-step?

---

- We can use [model.generate\(\)](#)
- There are two main advantages of `model.generate()`:
  1. We don't need to write a decoding loop on our own.
  2. We don't need to implement decoding strategies by ourselves.

# Pseudo code of a decoding loop

---

```
for step in range(max_length):
    # Forward pass through the model
    outputs = model.decoder(
        input_ids=decoder_input_ids,
        encoder_outputs=encoder_outputs
    )

    # Take the last hidden state's logits
    logits = outputs.logits[-1] # shape: (vocab_size,)

    # Apply softmax to get probabilities (optional, since argmax is invariant)
    probs = softmax(logits)

    # Select the token with the highest probability (greedy decoding)
    next_token_id = argmax(probs)

    # Append the predicted token to the decoder inputs
    decoder_input_ids.append(next_token_id)

    # If EOS token is generated, stop decoding
    if next_token_id == EOS_TOKEN_ID:
        break
```

# Perform Evaluations

---

Input:



Expected output of `model.generate()`:



Gold:



# Perform Evaluations Output Part

```
1 def do_evaluate(  
2     tokenizer,  
3     model,  
4     validation_loader,  
5     rouge_metric,  
6     inner_check=False,  
7 ):  
8     pbar = tqdm(validation_loader)  
9     pbar.set_description(f"Evaluating")  
10  
11     predictions = []  
12     references = []  
13     count = 0  
14     for ground_truth, inputs in pbar:  
15         output = [  
16             s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]  
17             for s in tokenizer.batch_decode(  
18                 model.generate(  
19                     **inputs,  
20                     max_new_tokens=200,  
21                     pad_token_id=tokenizer.eos_token_id,  
22                 )  
23             )  
24         ]
```

Expected output of `model.generate()`:

[CLS]

Source article

[SEP]

Summary

<|endoftext|>



# Decoding 後的處理 (接續上一頁)

```
output = [  
    s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]  
    for s in tokenizer.batch_decode(  
        model.generate(  
            **inputs,  
            max_new_tokens=200, # Maximum number of tokens to generate  
            pad_token_id=tokenizer.eos_token_id,  
        )  
    )  
]
```

s: [CLS] 中國足協今日宣布與國際足聯達成合作協議 [SEP] 中國足協與國際簽署協議 <|endoftext|>

s.split("[SEP]")[1]: 中國足協與國際足聯簽署協議 <|endoftext|>

.replace(" ", ""): 中國足協與國際足聯簽署協議<|endoftext|>

.split("<|endoftext|>")[0]: 中國足協與國際足聯簽署協議

# 通常我們會設置 max\_new\_tokens

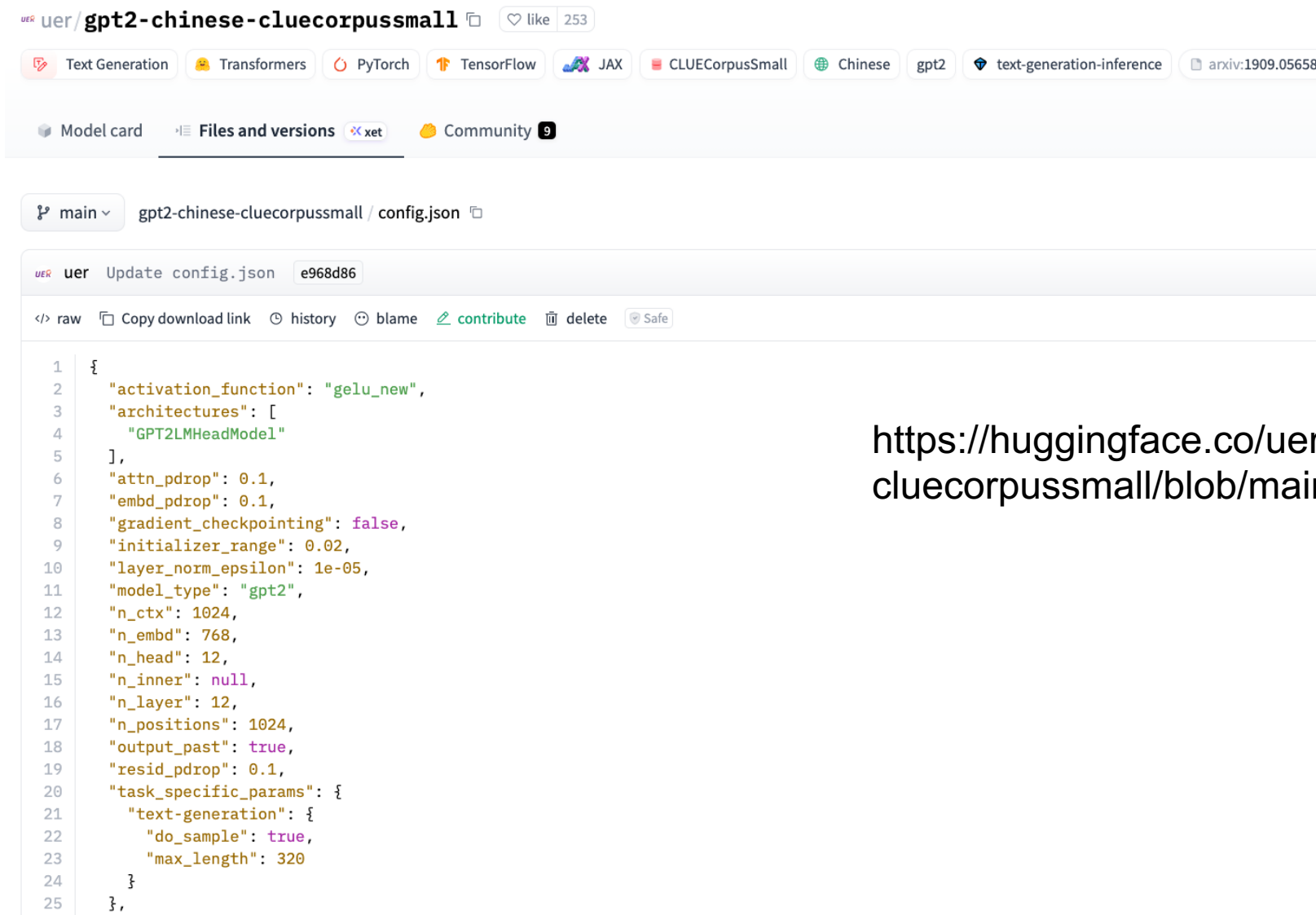
```
output = [  
    s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]  
    for s in tokenizer.batch_decode(  
        model.generate(  
            **inputs,  
            max_new_tokens=200, # Maximum number of tokens to generate  
            pad_token_id=tokenizer.eos_token_id,  
        )  
    )  
]
```

如果你不設 max\_new\_tokens，那模型 outputs 就會受到 max\_length 控制：



連 inputs 的部分都算進去 max\_length

# 查看 model config 最簡單的方式



The screenshot displays the Hugging Face interface for the model `uer/gpt2-chinese-cluecorpussmall`. The page includes a header with the model name, a like count of 253, and various tags such as Text Generation, Transformers, PyTorch, TensorFlow, JAX, CLUECorpusSmall, Chinese, gpt2, text-generation-inference, and arxiv:1909.05658. Below the header, there are tabs for Model card, Files and versions, and Community. The Files and versions tab is active, showing the `config.json` file. The file content is displayed in a code editor, showing the model's configuration parameters.

```
1 {
2   "activation_function": "gelu_new",
3   "architectures": [
4     "GPT2LMHeadModel"
5   ],
6   "attn_pdrop": 0.1,
7   "embd_pdrop": 0.1,
8   "gradient_checkpointing": false,
9   "initializer_range": 0.02,
10  "layer_norm_epsilon": 1e-05,
11  "model_type": "gpt2",
12  "n_ctx": 1024,
13  "n_embd": 768,
14  "n_head": 12,
15  "n_inner": null,
16  "n_layer": 12,
17  "n_positions": 1024,
18  "output_past": true,
19  "resid_pdrop": 0.1,
20  "task_specific_params": {
21    "text-generation": {
22      "do_sample": true,
23      "max_length": 320
24    }
25  },
```

<https://huggingface.co/uer/gpt2-chinese-cluecorpussmall/blob/main/config.json>

# 在 batch 生成時要用 EOS 去替代 Padding

```
output = [  
    s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]  
    for s in tokenizer.batch_decode(  
        model.generate(  
            **inputs,  
            max_new_tokens=200, # Maximum number of tokens to generate  
            pad_token_id=tokenizer.eos_token_id,  
        )  
    )  
]
```

原因：GPT-2 沒有 [PAD] token!

假設一個 batch 在生成時：

Sent1: 中國 足球 協會 今天 ... <|endoftext|>

Sent2: 美國 <|endoftext|> <|endoftext|> <|endoftext|> <|endoftext|> <|endoftext|>

# do\_evaluate 的 targets

```
25 targets = [  
26     s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")  
27     for s in tokenizer.batch_decode(ground_truth["input_ids"])  
28 ]  
29 assert len(output) == len(targets) 同「Decoding後的處理」那頁  
30 output = [" "] if output == [""] else output  
31 predictions.extend([" ".join(jieba.lcut(o)) for o in output])  
32 references.extend([" ".join(jieba.lcut(t)) for t in targets])  
33 count += 1  
34 if count > 100 and inner_check:  
35     break  
36  
37 score = rouge_metric.get_scores(predictions, references, avg=True)  
38 if inner_check:  
39     print("Validation using 100 examples: ", score)  
40 else:  
41     print(score)  
42  
43 return score, predictions, references
```

ground\_truth (對應 collate\_fn 的 complete\_text):

[CLS]

Source article

[SEP]

Summary

<|endoftext|>

# do\_evaluate 的 predictions

```
25     targets = [  
26         s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")  
27         for s in tokenizer.batch_decode(ground_truth["input_ids"])  
28     ]  
29     assert len(output) == len(targets)  
30     output = [" "] if output == [""] else output  
31     predictions.extend([" ".join(jieba.lcut(o)) for o in output])  
32     references.extend([" ".join(jieba.lcut(t)) for t in targets])  
33     count += 1  
34     if count > 100 and inner_check:  
35         break  
36  
37     score = rouge_metric.get_scores(predictions, references, avg=True)  
38     if inner_check:  
39         print("Validation using 100 examples: ", score)  
40     else:  
41         print(score)  
42  
43     return score, predictions, references
```

outputs 和 targets 現在都是文字

使用結巴斷詞後，才會計算 ROUGE（正確答案和預測的重疊率）

do\_evaluate  
的 score

```
25     targets = [  
26         s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")  
27         for s in tokenizer.batch_decode(ground_truth["input_ids"])  
28     ]  
29     assert len(output) == len(targets)  
30     output = [" "] if output == [""] else output  
31     predictions.extend([" ".join(jieba.lcut(o)) for o in output])  
32     references.extend([" ".join(jieba.lcut(t)) for t in targets])  
33     count += 1  
34     if count > 100 and inner_check:  
35         break  
36  
37     score = rouge_metric.get_scores(predictions, references, avg=True)  
38     if inner_check:  
39         print("Validation using 100 examples: ", score)  
40     else:  
41         print(score)  
42  
43     return score, predictions, references
```

avg=True: perform averaging for all the tested instances

# 完整的 Training Loop

```
1  step_i = 0
2  for epoch in range(NUM_EPOCHS):
3      pbar = tqdm(train_loader)
4      pbar.set_description(f"Training epoch [{epoch+1}/{NUM_EPOCHS}]")
5      for inputs, _ in pbar:
6          optimizer.zero_grad()
7          loss = model(**inputs).loss
8          loss.backward()
9          optimizer.step()
10         pbar.set_postfix(loss=loss.item())
11         writer.add_scalar("Loss/train", loss.item(), step_i)
12
13         if step_i % 1000 == 0 and step_i != 0:
14             score, pres, refs = do_evaluate(
15                 tokenizer=tokenizer,
16                 model=model,
17                 validation_loader=val_loader,
18                 rouge_metric=rouge_metric,
19                 inner_check=True,
20             )
21             print(f"Rouge scores on step{step_i} of epoch {epoch}:", score)
22             print("Predictions:", pres[:5])
23             writer.add_scalar("Rouge-1/val", score["rouge-1"] ["f"], step_i)
24             writer.add_scalar("Rouge-2/val", score["rouge-2"] ["f"], step_i)
25         step_i += 1
```



# Let's try it

---

1. 看一下你自己的 Wandb

# T5 [4]

[4] Raffel, Colin, et al. "Exploring the limits of transfer learning with a unified text-to-text transformer." *Journal of machine learning research* 21.140 (2020): 1-67.

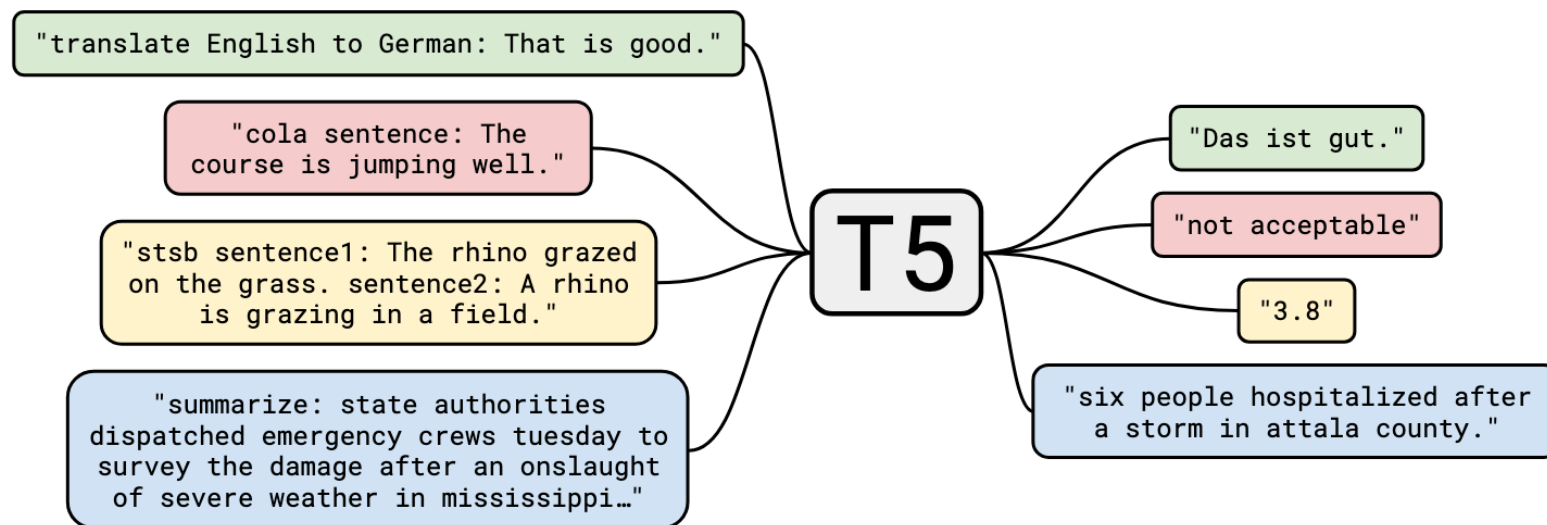
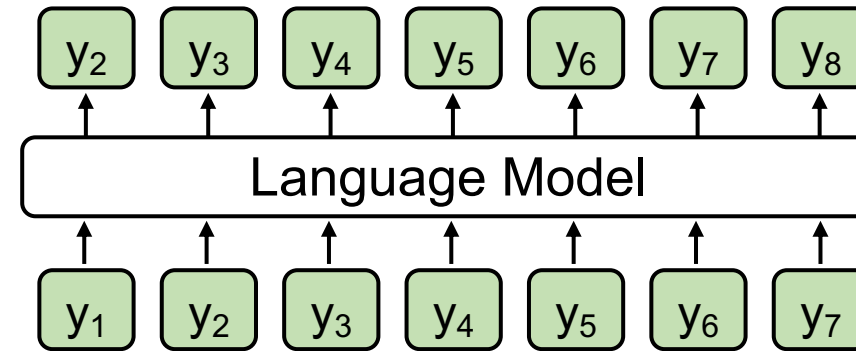


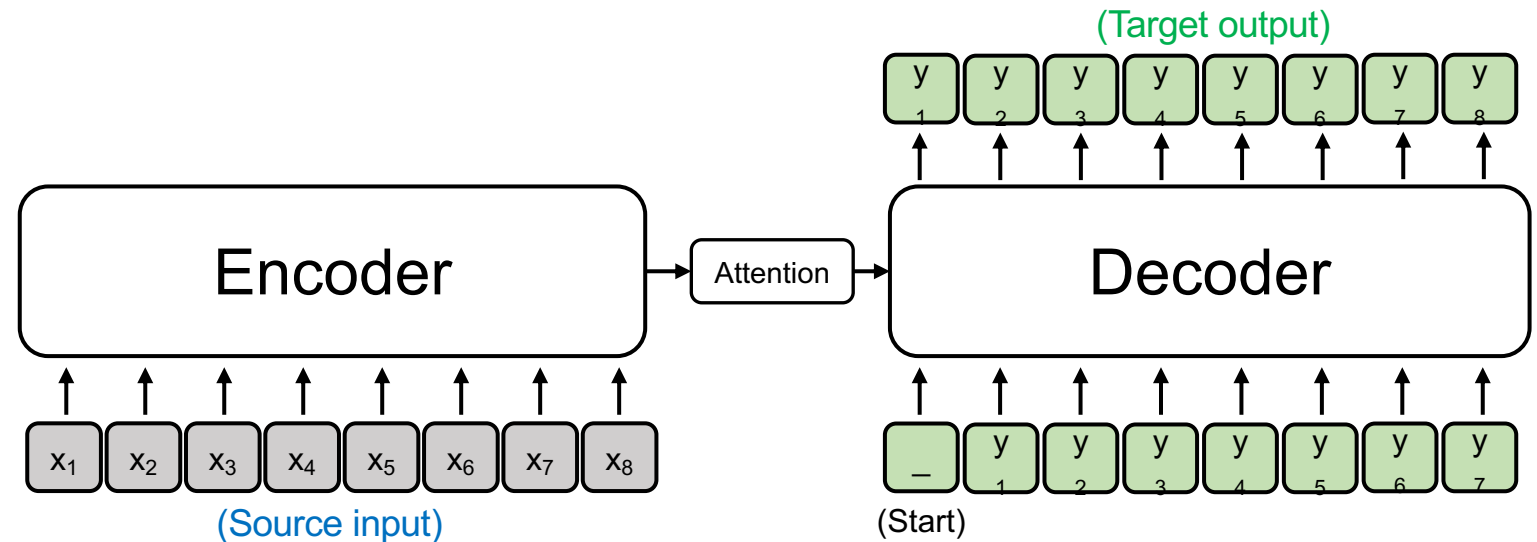
Figure 1: A diagram of our text-to-text framework. Every task we consider—including translation, question answering, and classification—is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks. It also provides a standard testbed for the methods included in our empirical survey. “T5” refers to our model, which we dub the “**T**ext-**t**o-**T**ext **T**ransfer **T**ransformer”.

# Language Model vs. Seq2seq Model

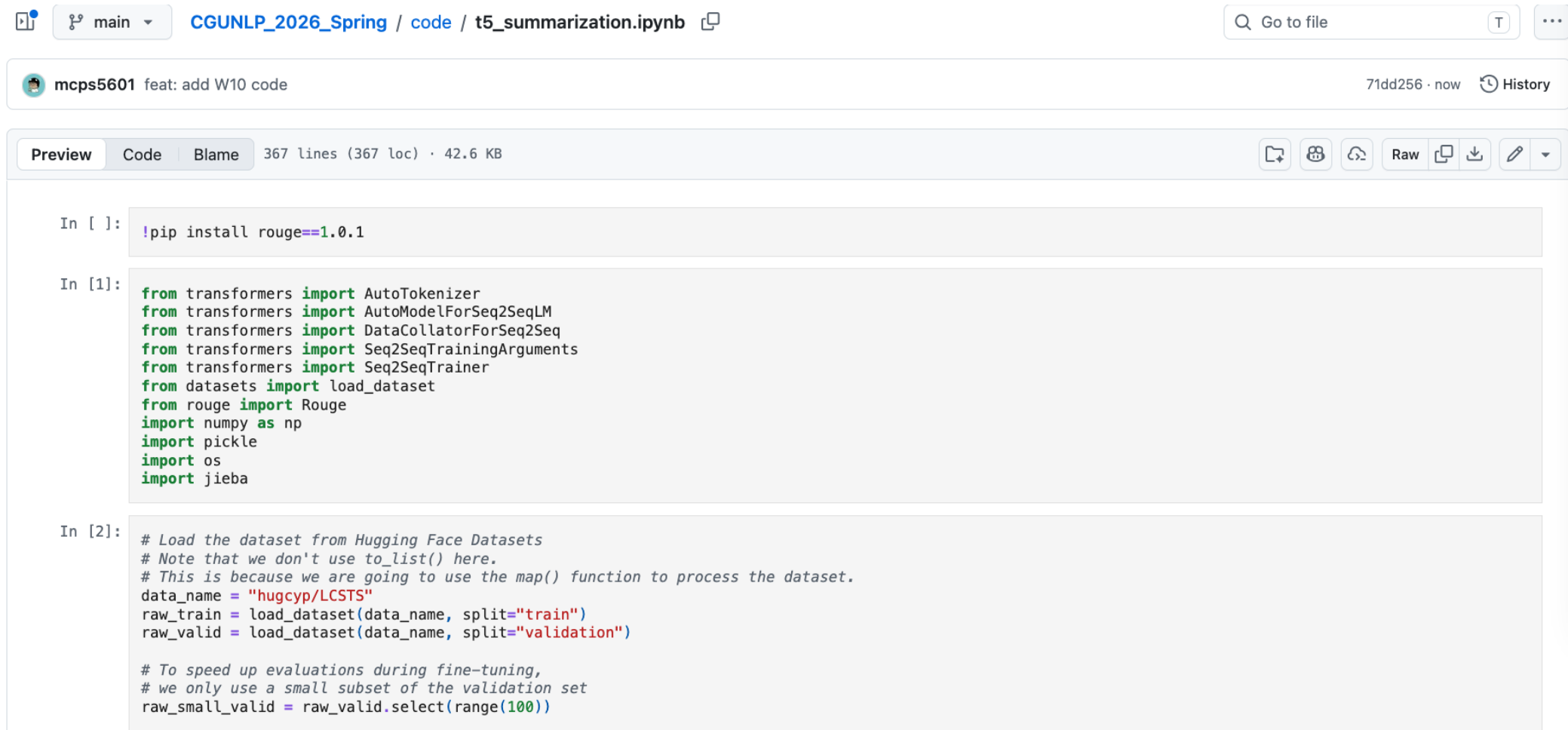
Casual Language Model  
(E.g., GPT)



Sequence to sequence Model  
(E.g., T5; Vanilla Transformers)



# See `code/t5\_summarization.ipynb`



```
In [ ]: !pip install rouge==1.0.1

In [1]: from transformers import AutoTokenizer
from transformers import AutoModelForSeq2SeqLM
from transformers import DataCollatorForSeq2Seq
from transformers import Seq2SeqTrainingArguments
from transformers import Seq2SeqTrainer
from datasets import load_dataset
from rouge import Rouge
import numpy as np
import pickle
import os
import jieba

In [2]: # Load the dataset from Hugging Face Datasets
# Note that we don't use to_list() here.
# This is because we are going to use the map() function to process the dataset.
data_name = "hugcyp/LCSTS"
raw_train = load_dataset(data_name, split="train")
raw_valid = load_dataset(data_name, split="validation")

# To speed up evaluations during fine-tuning,
# we only use a small subset of the validation set
raw_small_valid = raw_valid.select(range(100))
```

# Thank you!

Instructor: 林英嘉

 yjlin@cgu.edu.tw

TA: 林宣辰

 b1128015@cgu.edu.tw